

Lecture 3

A* Search and Search in Complex Environments

TDT4136: Introduction to Artificial Intelligence

Aleksandra Jekic

Based on slides by Xavier F. C. Sánchez Díaz

Department of Computer Science
Faculty of Information Technology and Electrical Engineering
Norwegian University of Science and Technology

September 3, 2026

Today we are covering the remaining part of Chapter 3 and all of Chapter 4.

Outline

1. Recap
2. A^* Search (Classic search)
3. Local Search (We do not care how we found the answer)
4. Local Search Algorithms
5. Nondeterministic, Partially Observable and Unknown Environments (Adding restrictions to search)

Recap on Search

- ▶ **Uninformed search strategies** systematically navigate the search space, with no knowledge of where the goal may be.
- ▶ Breadth-first search.
- ▶ Depth-first search.
- ▶ Depth-limited and Iterative deepening search.
- ▶ Uniform-cost search/Dijkstra.

Recap on Search

- ▶ **Uninformed search strategies** systematically navigate the search space, with no knowledge of where the goal may be.
- ▶ Breadth-first search.
- ▶ Depth-first search.
- ▶ Depth-limited and Iterative deepening search.
- ▶ Uniform-cost search/Dijkstra.
- ▶ **Informed search strategies** use a heuristic (guess estimates towards the goal).
- ▶ We covered Greedy Best First Search, where you pick the option with the best estimate
- ▶ We also covered A^* , which uses both the cost and the estimate

Recap on Search

Harder than it looks. Small details in the implementation can change results drastically. Online resources and earlier editions of the book cannot necessarily be relied upon.

- ▶ Is the algorithm checking for redundant paths (**graph search**) or not (**tree search**)?
- ▶ Is the goal check performed **early** (when a node is *generated*) or **late** (when a node is *expanded*)?
- ▶ Is the algorithm **storing** all **reached states**, or **reconstructing the path** from a chain of parent nodes?

Differences like these can entirely change the properties and guarantees of the algorithm. Out there, A* search can be many different things.

Read the book!

To become familiar with the algorithms and their implementations details, you should read the book. These slides are not a replacement for the book; they are a summary of the most important points.

Recap on Search

For **Breadth-First Search**, our main reference is the pseudocode found on page 95 in Norvig and Russell. For all the other algorithms, we use **Best-First Search** on Page 91 with the appropriate function f to order the priority queue.

Depth-First Search

$$f(n) = -\text{depth}(n)$$

Uniform-Cost Search (Dijkstra)

$$f(n) = g(n)$$

Greedy Best-First Search

$$f(n) = h(n)$$

A* Search

$$f(n) = g(n) + h(n)$$

Recap on Search

Breadth-First Search

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node  $\leftarrow$  NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier  $\leftarrow$  a FIFO queue, with node as an element
  reached  $\leftarrow$  {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    for each child in EXPAND(problem, node) do
      s  $\leftarrow$  child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

gn g
have I
explored

Recap on Search

Best-First Search

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure  
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)  
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element  
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then  
        reached[s]  $\leftarrow$  child  
        add child to frontier  
  return failure
```

```
function EXPAND(problem, node) yields nodes  
  s  $\leftarrow$  node.STATE  
  for each action in problem.ACTIONS(s) do  
    s'  $\leftarrow$  problem.RESULT(s, action)  
    cost  $\leftarrow$  node.PATH-COST + problem.ACTION-COST(s, action, s')  
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Recap on Search

Algorithm	Complete?	Optimal?	Time	Space
Breadth-First	Yes ¹	Yes ³	$O(b^d)$	$O(b^d)$
Uniform-Cost	Yes ^{1,2}	Yes	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Depth-First	No	No	$O(b^m)$	$O(bm)$
Depth-Limited	No	No	$O(b^\ell)$	$O(b\ell)$
Iterative Deepening	Yes ¹	Yes ³	$O(b^d)$	$O(bd)$

b : branching factor; d : depth of shallowest solution; m : maximum search depth; ℓ : depth limit.

1: Complete if b is finite, 2: Complete if all action costs are $\geq \epsilon > 0$, 3: Cost-optimal if action costs are all identical.

Common gotchas:

- Breadth-First Search has an early goal check, immediately when adding a new node to the frontier.
- For $g(n)$, you have to sum the full path to $g(n)$ while $h(n)$ just looks at the heuristic value at n .

A^* *optimality*

More on A^*

A^* is **cost-optimal** if **certain conditions are met**. What are these conditions?

A^* optimality

More on A^*

A^* is **cost-optimal** if **certain conditions are met**. What are these conditions?

- ▶ Arc costs need to be positive and a solution must exist.
- ▶ The heuristic function needs to be **admissible**.

Admissibility

More on A^*

Admissibility of h

We say a heuristic h is **admissible** if it never **overestimates** the cost from a node to the goal node.

Admissibility

More on A^*

Admissibility of h

We say a heuristic h is **admissible** if it **never overestimates** the cost from a node to the goal node.

An admissible heuristic means that for every node n :

- $h(n) \leq h^*(n)$, where $h^*(n)$ is the true distance.

Admissibility

More on A^*

Admissibility of h

We say a heuristic h is **admissible** if it **never overestimates** the cost from a node to the goal node.

An admissible heuristic means that for every node n :

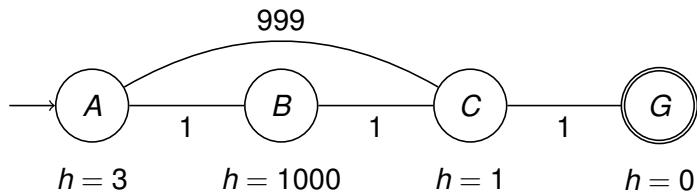
- $h(n) \leq h^*(n)$, where $h^*(n)$ is the true distance.

Note that if all heuristic values are 0, A^* collapses into UCS/Dijkstra which is cost-optimal.

An **admissible** heuristic is *optimistic*!

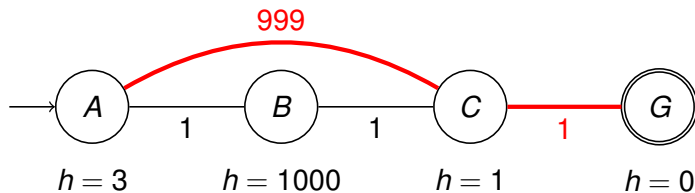
Essence of the proof

More on A^*



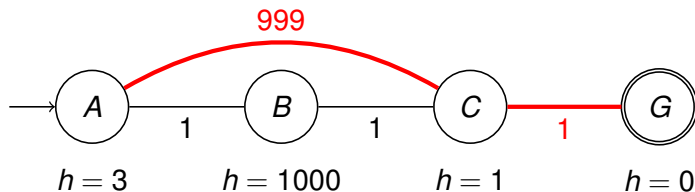
Essence of the proof

More on A^*



Essence of the proof

More on A^*



We would not choose the optimal path due to $h(B)$ being overestimated of the actual cost!

We have $h(B) = 1000 > h^*(B)$ which breaks admissibility, $h(n) \leq h^*(n)$.

Negative heuristics

An admissible heuristic means that for every node n :

- ▶ $h(n) \leq h^*(n)$, where $h^*(n)$ is the true distance.

Problems

- ▶ 1. Will A* be cost optimal if the heuristic costs can be negative?
- ▶ 2. What do negative heuristics represent?
- ▶ 3. Is this a good idea?

Negative heuristics

An admissible heuristic means that for every node n :

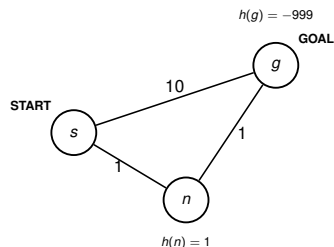
- ▶ $h(n) \leq h^*(n)$, where $h^*(n)$ is the true distance.

Problems

- ▶ 1. Will A* be cost optimal if the heuristic costs can be negative?
- ▶ 2. What do negative heuristics represent?
- ▶ 3. Is this a good idea?

Answers

- ▶ 1. Depends on whether we allow $h(goal) \neq 0$. Without this assumption, not cost optimal.
- ▶ 2. Since cost optimality requires positive path costs and the heuristics represent remaining distance, it does not really represent anything meaningful.
- ▶ 3. If you subtract a really big C from all heuristic values, it won't change the ordering. UCS is more informative than random negative values on average.



Consistency

More on A^*

Another important (and even stronger) property of a heuristic h is **consistency**.

Consistency of h

A heuristic h is **consistent** if for every node n and all of its successors n' generated by an action a , we have

$$h(n) \leq c(n, a, n') + h(n')$$

Consistency

More on A^*

Another important (and even stronger) property of a heuristic h is **consistency**.

Consistency of h

A heuristic h is **consistent** if for every node n and all of its successors n' generated by an action a , we have

$$h(n) \leq c(n, a, n') + h(n')$$

In other words, the estimate of a node should be less than or equal to the the estimate of a descendant plus the cost of reaching there.

Intuition: A heuristic should not overestimate the cost relative to later estimates.

Consistency: an example

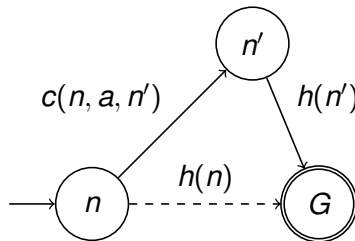
More on A^*

Consistency of h

A heuristic h is **consistent** if for every node n and all of its successors n' generated by an action a , we have

$$h(n) \leq c(n, a, n') + h(n')$$

- ▶ A triangle inequality helps picturing it!
- ▶ The sum of any two sides in a triangle must be greater than the third side.
- ▶ Moving through $h(n)$ **has** to be cheaper than going to G via the successor n'
- ▶ This **must** be true for every successor n' of n



Consistency and admissibility

More on A^*

Why is this important?

- ▶ A heuristic that is **consistent** is always **admissible**

Consistency and admissibility

More on A^*

Why is this important?

- ▶ A heuristic that is **consistent** is always **admissible**
 - ▶ Not necessarily the other way around!

Consistency and admissibility

More on A^*

Why is this important?

- ▶ A heuristic that is **consistent** is always **admissible**
 - ▶ Not necessarily the other way around!
- ▶ Since a consistent heuristic is admissible, then a consistent heuristic is also always **cost-optimal**

Consistency and admissibility

More on A^*

Why is this important?

- ▶ A heuristic that is **consistent** is always **admissible**
 - ▶ Not necessarily the other way around!
- ▶ Since a consistent heuristic is admissible, then a consistent heuristic is also always **cost-optimal**
- ▶ A popular version of A^* requires consistency. In the bonus task in Assignment 1 you are asked to find it and to consider its advantages.

Consistency and admissibility

More on A^*

Why is this important?

- ▶ A heuristic that is **consistent** is always **admissible**
 - ▶ Not necessarily the other way around!
- ▶ Since a consistent heuristic is admissible, then a consistent heuristic is also always **cost-optimal**
- ▶ A popular version of A^* requires consistency. In the bonus task in Assignment 1 you are asked to find it and to consider its advantages.
- ▶ A consistent heuristic $h(n)$ ensures that the cost function $f(n) = g(n) + h(n)$ is **monotonic nondecreasing**

Consistency and admissibility

More on A^*

Why is this important?

- ▶ A heuristic that is **consistent** is always **admissible**
 - ▶ Not necessarily the other way around!
- ▶ Since a consistent heuristic is admissible, then a consistent heuristic is also always **cost-optimal**
- ▶ A popular version of A^* requires consistency. In the bonus task in Assignment 1 you are asked to find it and to consider its advantages.
- ▶ A consistent heuristic $h(n)$ ensures that the cost function $f(n) = g(n) + h(n)$ is **monotonic nondecreasing**
 - ▶ That means that $f(n)$ is non-decreasing along any path

Optimality and efficiency

More on A^*

- ▶ A^* is **optimally efficient** with a **consistent** heuristic
- ▶ This means that any other search algorithm with the same heuristic values must at least expand all nodes that A^* expanded, ignoring ties.

Optimality and efficiency

More on A^*

- ▶ A^* is **optimally efficient** with a **consistent** heuristic
- ▶ This means that any other search algorithm with the same heuristic values must at least expand all nodes that A^* expanded, ignoring ties.

However, the main issue of A^* lies on its memory use. Some ways to reduce it:

- ▶ Reference count – remove a state from *reached* when there are no more ways to reach it

Optimality and efficiency

More on A^*

- ▶ A^* is **optimally efficient** with a **consistent** heuristic
- ▶ This means that any other search algorithm with the same heuristic values must at least expand all nodes that A^* expanded, ignoring ties.

However, the main issue of A^* lies on its memory use. Some ways to reduce it:

- ▶ Reference count – remove a state from *reached* when there are no more ways to reach it
- ▶ Beam search – limit size of *frontier* to k -best candidates

Optimality and efficiency

More on A^*

- ▶ A^* is **optimally efficient** with a **consistent** heuristic
- ▶ This means that any other search algorithm with the same heuristic values must at least expand all nodes that A^* expanded, ignoring ties.

However, the main issue of A^* lies on its memory use. Some ways to reduce it:

- ▶ Reference count – remove a state from *reached* when there are no more ways to reach it
- ▶ Beam search – limit size of *frontier* to k -best candidates
- ▶ Iterative deepening A^* – gradually increase the f -cost *cutoff*.

Optimality and efficiency

More on A^*

- ▶ A^* is **optimally efficient** with a **consistent** heuristic
- ▶ This means that any other search algorithm with the same heuristic values must at least expand all nodes that A^* expanded, ignoring ties.

However, the main issue of A^* lies on its memory use. Some ways to reduce it:

- ▶ Reference count – remove a state from *reached* when there are no more ways to reach it
- ▶ Beam search – limit size of *frontier* to k -best candidates
- ▶ Iterative deepening A^* – gradually increase the f -cost *cutoff*.
- ▶ Recursive best-first search A^* – Various algorithms that can back out of subtrees, remembering the best it saw. Risk of "thrashing".

A generalised heuristic search

More on A^*

Generalised heuristic search

$$f(n) = g(n) + w \cdot h(n)$$

where w is a *weight* defining how important the heuristic $h(n)$ is.

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.
- ▶ With $w = \infty$ you only care about the estimate

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.
- ▶ With $w = \infty$ you only care about the estimate
 - ▶ Choose the one that *seems* the cheapest

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.
- ▶ With $w = \infty$ you only care about the estimate
 - ▶ Choose the one that *seems* the cheapest
 - ▶ This is **Greedy Best-First search**

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.
- ▶ With $w = \infty$ you only care about the estimate
 - ▶ Choose the one that *seems* the cheapest
 - ▶ This is **Greedy Best-First search**
- ▶ with $w = 1$ you care equally about the path cost and the estimates

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.
- ▶ With $w = \infty$ you only care about the estimate
 - ▶ Choose the one that *seems* the cheapest
 - ▶ This is **Greedy Best-First search**
- ▶ with $w = 1$ you care equally about the path cost and the estimates
 - ▶ This is A^*

A generalised heuristic search

More on A^*

$$f(n) = g(n) + w \cdot h(n)$$

- ▶ With $w = 0$ you only care about the cost of the path
 - ▶ Choose the cheapest!
 - ▶ This is called **uniform-cost search** and it's an uninformed search.
 - ▶ It is also known as **Dijkstra's algorithm**.
- ▶ With $w = \infty$ you only care about the estimate
 - ▶ Choose the one that *seems* the cheapest
 - ▶ This is **Greedy Best-First search**
- ▶ with $w = 1$ you care equally about the path cost and the estimates
 - ▶ This is A^*
- ▶ If you set $w > 1$, you get weighted A^* search which can be called "somewhat-greedy search". Can be faster (if lunch happens to be free today).

Building heuristics

More on A^*



How far are we from solving this sliding puzzle?

- ▶ $h_1(n)$ will be the number of misplaced tiles
- ▶ $h_2(n)$ will be the **total Manhattan** distance, the sum number of squares each tile is away from its desired location.

Building heuristics

More on A^*



How far are we from solving this sliding puzzle?

- ▶ $h_1(n)$ will be the number of misplaced tiles
- ▶ $h_2(n)$ will be the **total Manhattan** distance, the sum number of squares each tile is away from its desired location.

Any placement of the tiles is a state.

Building heuristics is a discipline itself

More on A^*

- ▶ Both $h_1(n)$ and $h_2(n)$ were essentially **relaxations** of the problem.
 - ▶ A tile can move from square X to square Y if X is adjacent to Y and Y is blank.
 - ▶ $h_2(n)$ removes blankness restriction, $h_1(n)$ removes both blankness and adjacency requirement.
- ▶ Consider dividing into subproblems.
 - ▶ For example, instead of solving the whole sliding puzzle at once, consider getting in place 1, 2, 3, 4.
 - ▶ Store all these solutions in a DB. Create an admissible heuristic from how close to correct it is.
 - ▶ You can solve multiple subproblems and combine them into a heuristic.
 - ▶ Computation and memory costs blur the line with having a pre-computed database of solutions.

Dominance: comparing heuristics

More on A^*

Which of the heuristics is better?

Dominance: comparing heuristics

More on A^*

Which of the heuristics is better?

Admissible heuristics can be compared by looking at their values.

Heuristic Domination

An admissible heuristic h_2 it is said to **dominate** another admissible heuristic h_1 if for every node n , $h_2(n) \geq h_1(n)$.

This will reflect in A^* expanding fewer nodes on h_2 , and thus finding an optimal solution, faster.

Dominance: comparing heuristics

More on A^*

Which of the heuristics is better?

Admissible heuristics can be compared by looking at their values.

Heuristic Domination

An admissible heuristic h_2 it is said to **dominate** another admissible heuristic h_1 if for every node n , $h_2(n) \geq h_1(n)$.

This will reflect in A^* expanding fewer nodes on h_2 , and thus finding an optimal solution, faster.

A generalisation of this would then be

$$h_{best}(n) = \max(h_a(n), h_b(n), \dots)$$

Dominance: comparing heuristics

More on A^*

Heuristic Domination

An admissible heuristic h_2 it is said to **dominate** another admissible heuristic h_1 if for every node n , $h_2(n) \geq h_1(n)$.

This will reflect in A^* expanding fewer nodes on h_2 , and thus finding an optimal solution, faster.

A simple intuition for this is that the heuristic with the greatest acceptable numbers contains the most information about the problem.

Considering the sliding puzzle, $h_2(n)$ dominates $h_1(n)$ because it only removes one restriction. On page 117 of the book, you can see a nice table showing how $h_2(n)$ consistently outperforms $h_1(n)$.

Note that we are crossing over into Chapter 4, Search in Complex environments now.

Search in Complex Environments

- ▶ Local Search
- ▶ Local Search Algorithms
- ▶ Nondeterministic, Partially Observable and Unknown Environments

Section 3

Local Search

Local Search and Optimization

- ▶ Both informed and uninformed searching strategies are designed to explore search spaces systematically
- ▶ They keep one or more paths in memory, and record which alternatives have been explored at each point along the path
- ▶ The path to that goal constitutes a solution
- ▶ But in most problems in the real world, the path to a solution **might be irrelevant**
- ▶ For example, I only care about what prime numbers were multiplied to get a certain very large number, not all the numbers we tried to find them.

Local Search and Optimization

- ▶ Both informed and uninformed searching strategies are designed to explore search spaces systematically
- ▶ They keep one or more paths in memory, and record which alternatives have been explored at each point along the path
- ▶ The path to that goal constitutes a solution
- ▶ But in most problems in the real world, the path to a solution **might be irrelevant**
- ▶ For example, I only care about what prime numbers were multiplied to get a certain very large number, not all the numbers we tried to find them.

If we only care about finding a solution, then there are better ways to search the space!

Local Search and Optimization

- ▶ It uses a single **current node** and moves to **neighbouring** nodes

Local Search and Optimization

- ▶ It uses a single **current node** and moves to **neighbouring** nodes
- ▶ It eases up on the completeness and optimality in the interest of improving time and space complexity.

Local Search and Optimization

- ▶ It uses a single **current node** and moves to **neighbouring** nodes
- ▶ It eases up on the completeness and optimality in the interest of improving time and space complexity.
- ▶ Local Search algorithms use “little” memory (usually a constant amount).

Local Search and Optimization

- ▶ It uses a single **current node** and moves to **neighbouring** nodes
- ▶ It eases up on the completeness and optimality in the interest of improving time and space complexity.
- ▶ Local Search algorithms use “little” memory (usually a constant amount).
- ▶ They can often find reasonable solutions in very large (or infinite) state spaces.

Local Search and Optimization

- ▶ It uses a single **current node** and moves to **neighbouring** nodes
- ▶ It eases up on the completeness and optimality in the interest of improving time and space complexity.
- ▶ Local Search algorithms use “little” memory (usually a constant amount).
- ▶ They can often find reasonable solutions in very large (or infinite) state spaces.
- ▶ Note the distinction between **continuous optimization problems** and **discrete optimization problems**. Many concepts overlap, but they have to be handled differently.

The search landscape

Local Search and Optimization

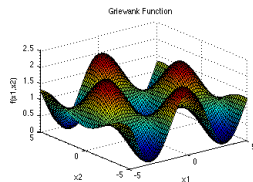
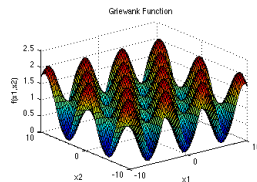
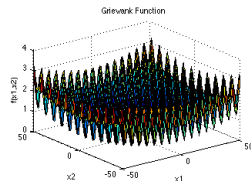
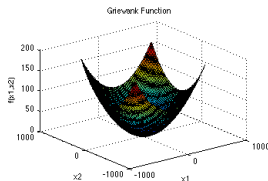
Usually, the **state space** is referred to as the **search space**. We can **visualise** this space. What is the minimum of this function?

$$f(x) = \sum_{i=1}^d \frac{x_i^2}{4000} - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$$

This idea is very important in artificial intelligence and optimization. Well worth understanding.

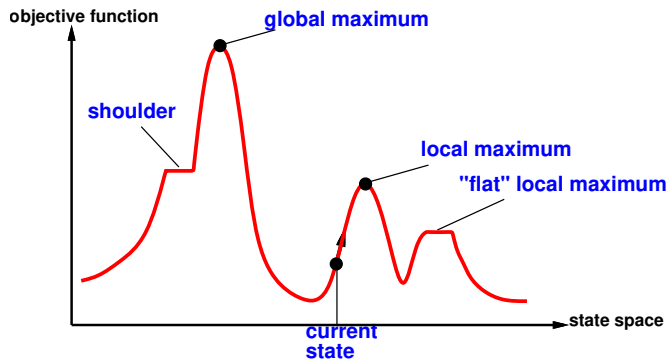
The Griewank function. Image from Surjanovic & Bingham

<https://www.sfu.ca/~ssurjano/griewank.html>



The search landscape

Local Search and Optimization

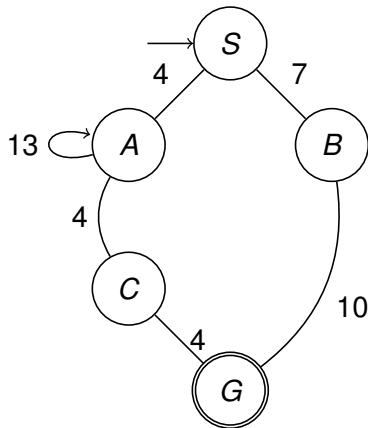


- ▶ Each point in the landscape represents a state in the search space and has “an elevation”
- ▶ If the elevation corresponds to an objective function, then the aim is to find the highest peak (or **maximum**)
- ▶ If the elevation corresponds to a cost function, then we look for the lowest valley (or **minimum**)

The search landscape

Local Search and Optimization

Recall our search problems.



- ▶ A is a neighbour of S, C and itself because those are the states than can be reached from A.
- ▶ The **neighbourhood** of A is then $\{A, C, S\}$.
- ▶ This concept of **neighbourhood** is very important for **local search**, as we decide *where to move next* by looking around us!

Section 4

Local Search Algorithms

As the last time with algorithms, please check
the full details in the book!

Hill climbing and gradient descent

Local search algorithms

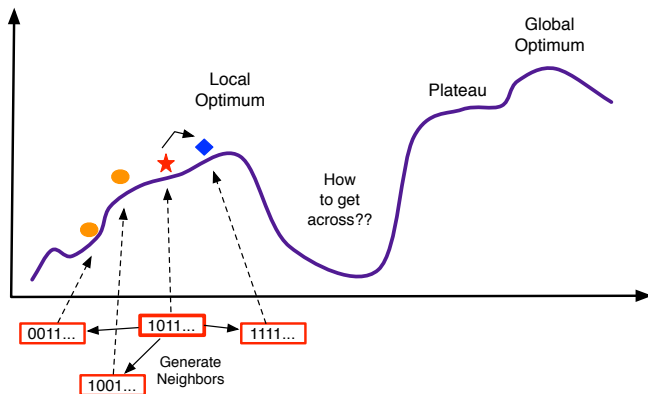
Idea: Go to the best spot you see now.

Hill climbing and gradient descent

Local search algorithms

Idea: Go to the best spot you see now.

- Assume you are doing **maximisation**
- You then want to **climb** the tallest peak
- This is called **hill-climbing**!



If you are **minimising** instead, then the procedure is called **gradient descent** as we want to move towards the direction where the difference in “height” is largest.

How to get across?

Both **hill-climbing** and **gradient descent** get stuck in **local optima**. How do we get out of this mess?

How to get across?

Both **hill-climbing** and **gradient descent** get stuck in **local optima**. How do we get out of this mess?

- ▶ Idea: take some *not so good* decisions every now and then!
 - ▶ This is what we call **stochastic local search**.

How to get across?

Both **hill-climbing** and **gradient descent** get stuck in **local optima**. How do we get out of this mess?

- ▶ Idea: take some *not so good* decisions every now and then!
 - ▶ This is what we call **stochastic local search**.
- ▶ Idea 2: Make it so that you gradually reduce the frequency of taking such “bad” decisions
 - ▶ This is the key to the **simulated annealing** algorithm
 - ▶ The idea is how atoms move around a lot with heat and settle as temperature cools.
 - ▶ Uses a physics formula to determine whether to accept new states based on temperature, $P(\text{accept}) = e^{-\Delta E/T}$.
 - ▶ If the temperature cools sufficiently slowly, the probability of finding the global minimum approaches 1.
 - ▶ Proteins in real life do something similar to find their optimal configuration.

How to get across?

Both **hill-climbing** and **gradient descent** get stuck in **local optima**. How do we get out of this mess?

- ▶ Idea: take some *not so good* decisions every now and then!
 - ▶ This is what we call **stochastic local search**.
- ▶ Idea 2: Make it so that you gradually reduce the frequency of taking such “bad” decisions
 - ▶ This is the key to the **simulated annealing** algorithm
 - ▶ The idea is how atoms move around a lot with heat and settle as temperature cools.
 - ▶ Uses a physics formula to determine whether to accept new states based on temperature, $P(\text{accept}) = e^{-\Delta E/T}$.
 - ▶ If the temperature cools sufficiently slowly, the probability of finding the global minimum approaches 1.
 - ▶ Proteins in real life do something similar to find their optimal configuration.
- ▶ Idea 3: Search multiple paths in batches
 - ▶ This is the key idea behind **population-based optimisation**

This line of research is usually referred to as **metaheuristics**.

Local search application: Machine learning

Hopefully I won't get in trouble for this, but everything is connected.

Local search application: Machine learning

Hopefully I won't get in trouble for this, but everything is connected.

- ▶ Suppose we have a 1 billion parameter function (a neural network) which takes a question (input) and returns an answer (output). Making text into numbers is trivial.
- ▶ We can count how many times the 1 billion parameter function misses the mark on the training data with questions and answers from books (a loss function).
- ▶ With the loss function and adjustable parameters we have a 1 billion-dimensional search landscape! How do we find the parameters that give the lowest error (landscape bottom)?
- ▶ From Calculus we know that $f'(x)$ tells us in which direction a function grows the most. For more than one dimension, it is $\nabla f(x)$.
- ▶ Now we just move the function in the opposite direction of growth, $-\nabla_{\theta} f(x)$, with regards to the parameters. This is local search (specifically gradient descent). We also jump around a bit to avoid getting stuck.

Local search application: Machine learning

Hopefully I won't get in trouble for this, but everything is connected.

- ▶ Suppose we have a 1 billion parameter function (a neural network) which takes a question (input) and returns an answer (output). Making text into numbers is trivial.
- ▶ We can count how many times the 1 billion parameter function misses the mark on the training data with questions and answers from books (a loss function).
- ▶ With the loss function and adjustable parameters we have a 1 billion-dimensional search landscape! How do we find the parameters that give the lowest error (landscape bottom)?
- ▶ From Calculus we know that $f'(x)$ tells us in which direction a function grows the most. For more than one dimension, it is $\nabla f(x)$.
- ▶ Now we just move the function in the opposite direction of growth, $-\nabla_{\theta} f(x)$, with regards to the parameters. This is local search (specifically gradient descent). We also jump around a bit to avoid getting stuck.
- ▶ A few weeks later, and several rivers drained, we have created ChatGPT! We are beyond redemption forever now. Even our mothers will not love us anymore.
- ▶ Note that for this problem, we do not care about what steps we took to get to the bottom of the landscape. (We also have no idea why it works or how reliable it is.)

Local search application: Machine learning

Now suppose someone sabotaged your training data with lots of mistakes so the landscape is basically incredibly misleading. What do we do now?

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

1. Start with a population of k randomly generated states

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

1. Start with a population of k randomly generated states
2. Randomly choose two parent states weighted by their fitness (objective function)

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

1. Start with a population of k randomly generated states
2. Randomly choose two parent states weighted by their fitness (objective function)
3. Generate child states by combining parent states randomly

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

1. Start with a population of k randomly generated states
2. Randomly choose two parent states weighted by their fitness (objective function)
3. Generate child states by combining parent states randomly
4. Add child states to the population

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

1. Start with a population of k randomly generated states
2. Randomly choose two parent states weighted by their fitness (objective function)
3. Generate child states by combining parent states randomly
4. Add child states to the population
5. Replace the old population by the new

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

1. Start with a population of k randomly generated states
2. Randomly choose two parent states weighted by their fitness (objective function)
3. Generate child states by combining parent states randomly
4. Add child states to the population
5. Replace the old population by the new

Genetic Algorithms

Local search algorithms

A well-known metaheuristic in the family of **population-based** optimisers is the **genetic algorithm**. If you have no signal on where to go, this can be very powerful.

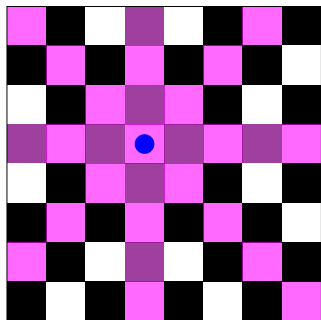
1. Start with a population of k randomly generated states
2. Randomly choose two parent states weighted by their fitness (objective function)
3. Generate child states by combining parent states randomly
4. Add child states to the population
5. Replace the old population by the new

This process will be repeated until a solution has been found, or until enough *generations* have been replaced.

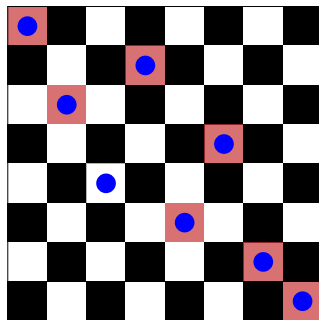
We have a whole course on evolutionary computation methods during the spring semester:
IT3708 Bio-Inspired AI!

The 8-queens problem

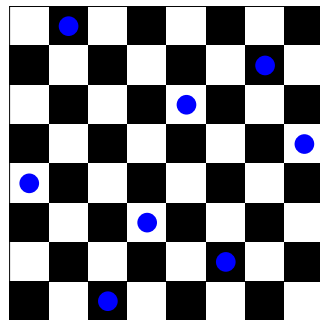
Place 8 queens in a chess board such that no queen checks each other.



(a) Queen constraints



(b) Generated conflicts



(c) Possible solution

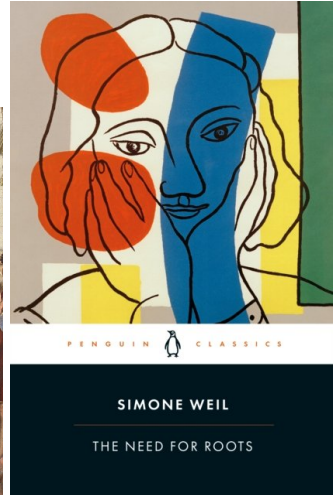
Figure: The 8-queens problem. 1a shows the constraints (in pink) imposed by the placement of a single queen piece (in blue). 1b highlights the conflicts arising from a possible configuration of the board. 1c illustrates one possible solution with no conflicts.

See a worked example in https://ntnu-ai-lab.github.io/EvoLP.jl/stable/tuto/8_queens.html

Genetic Algorithms

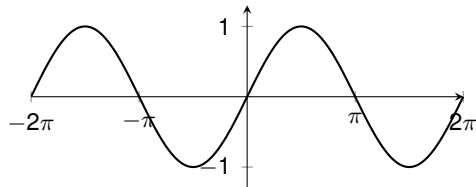
Well-known application

Nature accidentally did something like this one time, using survival as a fitness function, and now we are here and nobody is quite sure what to do.



Wake up problem

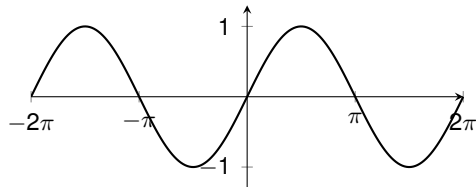
$$f(x) = \sin(x)$$



You want to find the minimum. Do you think gradient descent with $-f'(x)$ or genetic algorithms are faster? Assume representing this in a genetic algorithm is trivial.

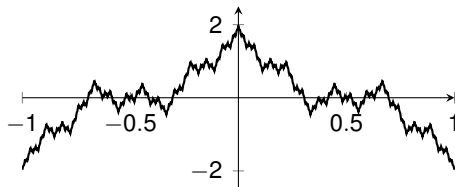
Wake up problem

$$f(x) = \sin(x)$$



You want to find the minimum. Do you think gradient descent with $-f'(x)$ or genetic algorithms are faster? Assume representing this in a genetic algorithm is trivial.

$$W(x) = \sum_{n=0}^{\infty} a^n \cos(b^n \pi x)$$



Now you want to find the minimum of the Weierstrass function. What is faster now?

Section 5

Nondeterministic, partially observable and unknown environments

Searching with Nondeterminism

- ▶ So far, we have assumed that actions are deterministic

Searching with Nondeterminism

- ▶ So far, we have assumed that actions are deterministic
 - ▶ That our intended action will **always** yield the result we expect

Searching with Nondeterminism

- ▶ So far, we have assumed that actions are deterministic
 - ▶ That our intended action will always yield the result we expect
- ▶ In the real-world, things do not always go as expected

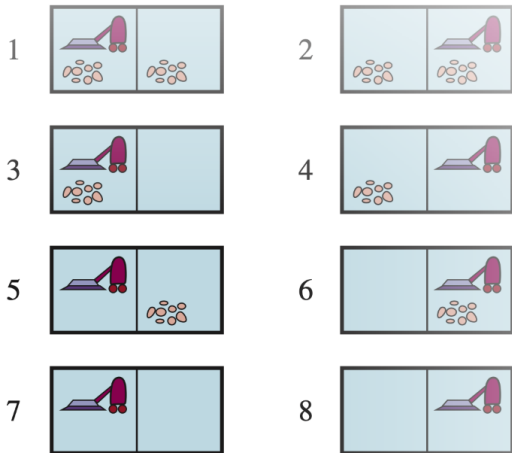
Searching with Nondeterminism

- ▶ So far, we have assumed that actions are deterministic
 - ▶ That our intended action will always yield the result we expect
- ▶ In the real-world, things do not always go as expected
- ▶ To account for different possible outcomes, we need to come up with a contingency plan instead of a single path of actions

Example: The Erratic Vacuum World

Searching with Nondeterminism

Suck *sometimes* cleans square. *Sometimes* also cleans both squares. If square is already clean, *sometimes* deposits dirt instead.

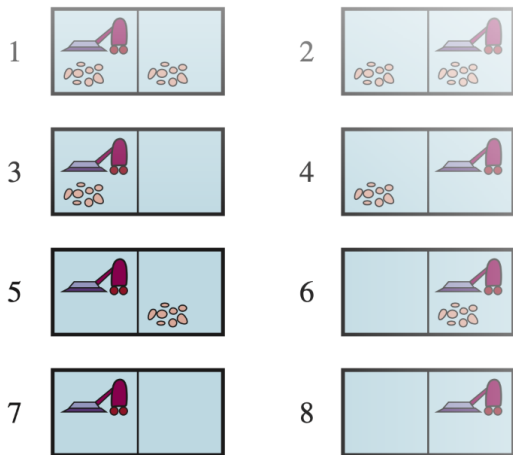


► Nondeterministic *suck* action:
 $suck(s_1) = \{s_5, s_7\}$

Example: The Erratic Vacuum World

Searching with Nondeterminism

Suck *sometimes* cleans square. *Sometimes* also cleans both squares. If square is already clean, *sometimes* deposits dirt instead.

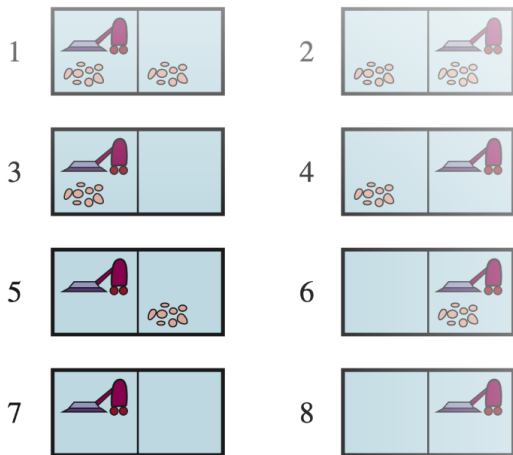


- Nondeterministic *suck* action:
 $suck(s_1) = \{s_5, s_7\}$
 - which means both states s_5 and s_7 are possible outcomes of executing a *suck* action on state s_1

Example: The Erratic Vacuum World

Searching with Nondeterminism

Suck *sometimes* cleans square. *Sometimes* also cleans both squares. If square is already clean, *sometimes* deposits dirt instead.

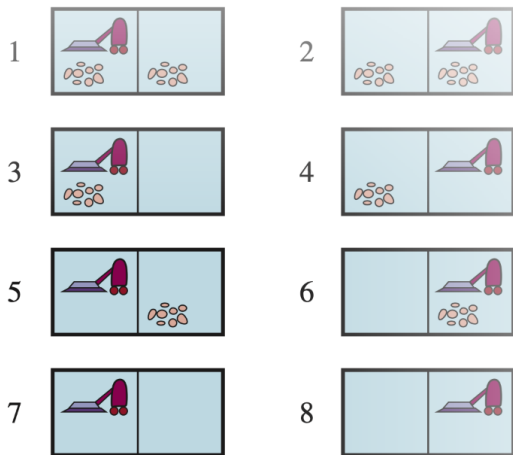


- Nondeterministic *suck* action:
 $suck(s_1) = \{s_5, s_7\}$
 - which means both states s_5 and s_7 are possible outcomes of executing a *suck* action on state s_1
- $suck(s_7) = \{s_3, s_7\}$

Example: The Erratic Vacuum World

Searching with Nondeterminism

Suck *sometimes* cleans square. *Sometimes* also cleans both squares. If square is already clean, *sometimes* deposits dirt instead.

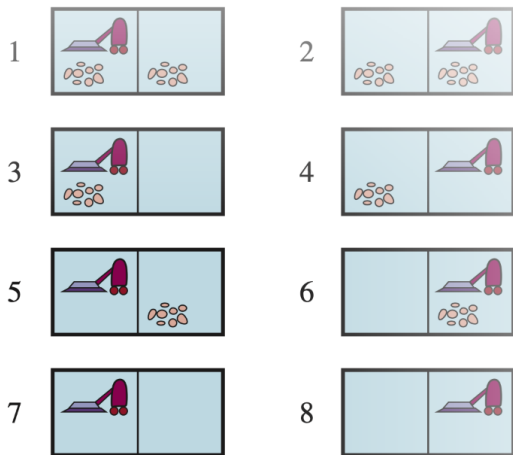


- ▶ Nondeterministic *suck* action:
 $suck(s_1) = \{s_5, s_7\}$
 - ▶ which means both states s_5 and s_7 are possible outcomes of executing a *suck* action on state s_1
- ▶ $suck(s_7) = \{s_3, s_7\}$
 - ▶ Which means both s_3 and s_7 are possible outcomes of *suck* on s_7

Example: The Erratic Vacuum World

Searching with Nondeterminism

Suck *sometimes* cleans square. *Sometimes* also cleans both squares. If square is already clean, *sometimes* deposits dirt instead.

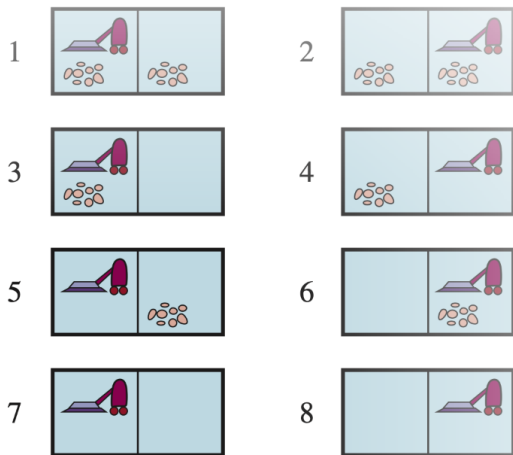


- ▶ Nondeterministic *suck* action:
 $suck(s_1) = \{s_5, s_7\}$
 - ▶ which means both states s_5 and s_7 are possible outcomes of executing a *suck* action on state s_1
- ▶ $suck(s_7) = \{s_3, s_7\}$
 - ▶ Which means both s_3 and s_7 are possible outcomes of *suck* on s_7

Example: The Erratic Vacuum World

Searching with Nondeterminism

Suck *sometimes* cleans square. *Sometimes* also cleans both squares. If square is already clean, *sometimes* deposits dirt instead.



- ▶ Nondeterministic *suck* action:
 $suck(s_1) = \{s_5, s_7\}$
 - ▶ which means both states s_5 and s_7 are possible outcomes of executing a *suck* action on state s_1
- ▶ $suck(s_7) = \{s_3, s_7\}$
 - ▶ Which means both s_3 and s_7 are possible outcomes of *suck* on s_7

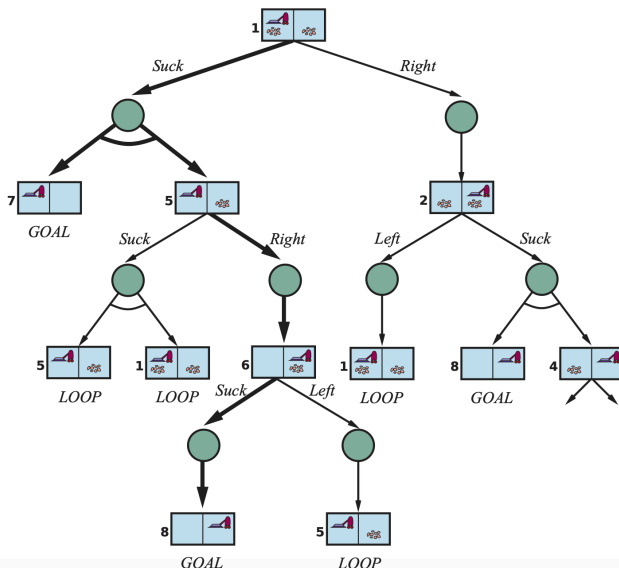
Nondeterminism can happen with other actions like *moveRight*! See the *slippery vacuum world* in the book!

AND-OR search trees

Searching with Nondeterminism

One way to handle these, is to consider *compound nodes*, made up of the possible states after a given action

- **OR** nodes represent **actions** (rectangles)
- **AND** nodes represent **outcomes** (circles)
- Since it is a **tree**, we can **search** in it
 - This is called **AND-OR search**
 - It is recursive, with a base case of either failure or an empty plan



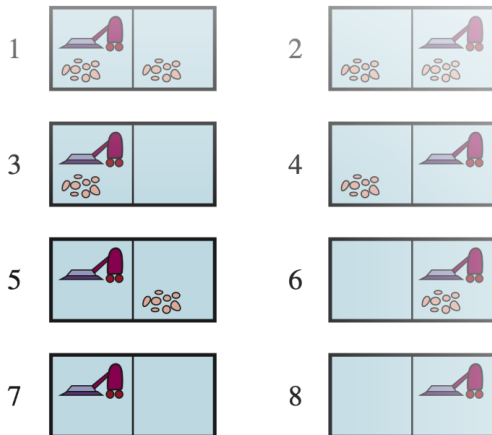
Searching in Partially Observable Environments

- ▶ So far, we have assumed that the agent knows exactly the state of its environment
- ▶ In reality, an agent receives partial (and possibly noisy) observations
- ▶ Therefore, the state can only be *estimated* through a “belief”

Sensorless deterministic vacuum world

Searching in partially observable environments

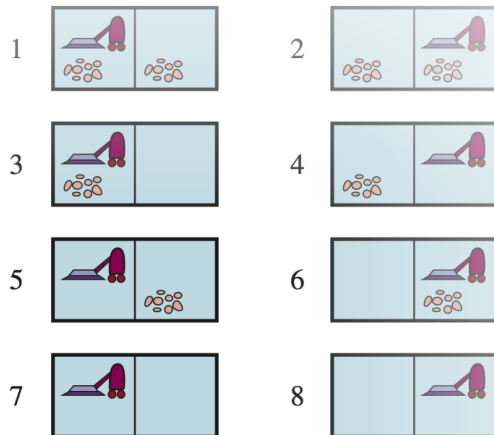
► $Result(\{1, 2, 3, 4, 5, 6, 7, 8\}, moveRight) = \{2, 4, 6, 8\}$



Sensorless deterministic vacuum world

Searching in partially observable environments

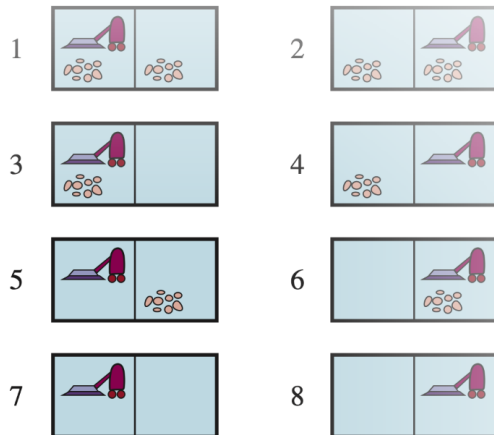
- ▶ $Result(\{1, 2, 3, 4, 5, 6, 7, 8\}, moveRight) = \{2, 4, 6, 8\}$
 - ▶ Which means that executing *moveRight* on any state $s \in S$ will yield a result in $\{2, 4, 6, 8\}$



Sensorless deterministic vacuum world

Searching in partially observable environments

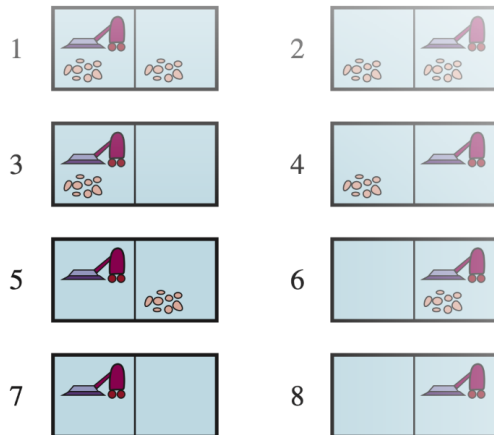
- ▶ $Result(\{1, 2, 3, 4, 5, 6, 7, 8\}, moveRight) = \{2, 4, 6, 8\}$
 - ▶ Which means that executing *moveRight* on any state $s \in S$ will yield a result in $\{2, 4, 6, 8\}$
- ▶ $Result(\{2, 4, 6, 8\}, Suck) = \{4, 8\}$



Sensorless deterministic vacuum world

Searching in partially observable environments

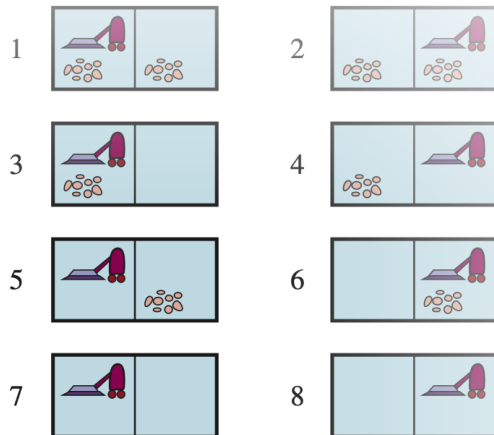
- ▶ $Result(\{1, 2, 3, 4, 5, 6, 7, 8\}, moveRight) = \{2, 4, 6, 8\}$
 - ▶ Which means that executing *moveRight* on any state $s \in S$ will yield a result in $\{2, 4, 6, 8\}$
- ▶ $Result(\{2, 4, 6, 8\}, Suck) = \{4, 8\}$
- ▶ $Result(\{4, 8\}, Left) = \{3, 7\}$



Sensorless deterministic vacuum world

Searching in partially observable environments

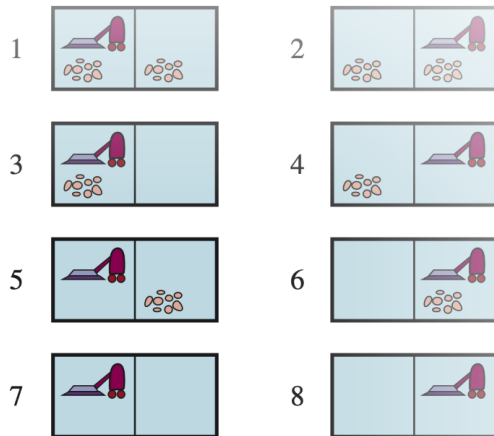
- ▶ $Result(\{1, 2, 3, 4, 5, 6, 7, 8\}, moveRight) = \{2, 4, 6, 8\}$
 - ▶ Which means that executing *moveRight* on any state $s \in S$ will yield a result in $\{2, 4, 6, 8\}$
- ▶ $Result(\{2, 4, 6, 8\}, Suck) = \{4, 8\}$
- ▶ $Result(\{4, 8\}, Left) = \{3, 7\}$
- ▶ $Result(\{3, 7\}, Suck) = \{7\}$



Sensorless deterministic vacuum world

Searching in partially observable environments

- ▶ $Result(\{1, 2, 3, 4, 5, 6, 7, 8\}, moveRight) = \{2, 4, 6, 8\}$
 - ▶ Which means that executing $moveRight$ on any state $s \in S$ will yield a result in $\{2, 4, 6, 8\}$
- ▶ $Result(\{2, 4, 6, 8\}, Suck) = \{4, 8\}$
- ▶ $Result(\{4, 8\}, Left) = \{3, 7\}$
- ▶ $Result(\{3, 7\}, Suck) = \{7\}$

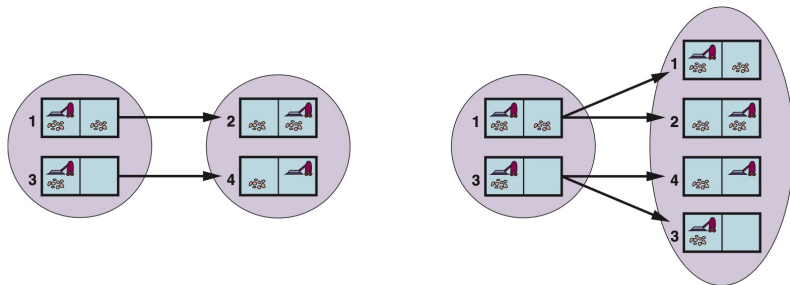


Think of 5D-chess: you solve the problem on multiple paths at the same time!

Predicting the next state with sensorless agents

Searching in partially observable environments

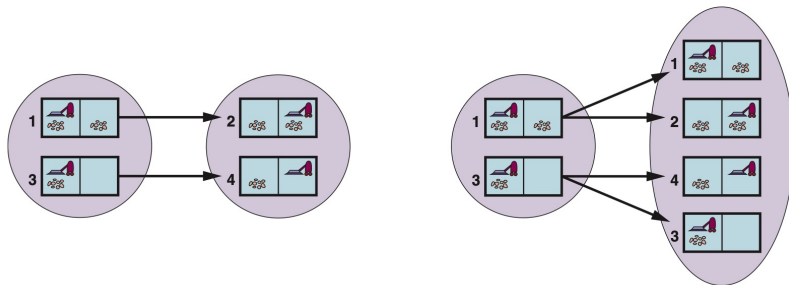
We are, in a way, making compound nodes with multiple outcomes in, where some of our actions lead to specific environment settings inside those belief states.



Predicting the next state with sensorless agents

Searching in partially observable environments

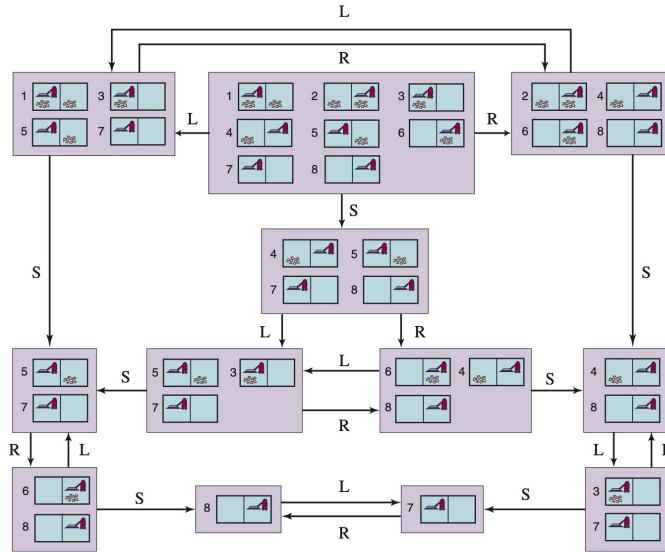
We are, in a way, making compound nodes with multiple outcomes in, where some of our actions lead to specific environment settings inside those belief states.



Of course it can be **both** nondeterministic and partially observable!

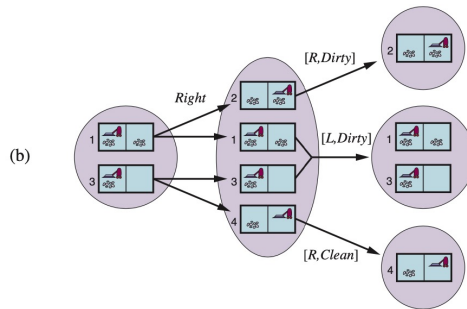
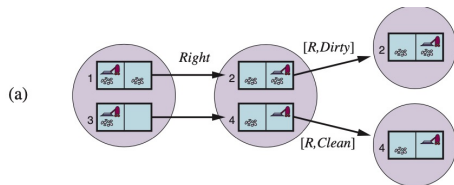
Searching through the belief space in deterministic environments

If we have a deterministic setting, we can use an ordinary search algorithm.



Searching through the belief space in partially observable environments

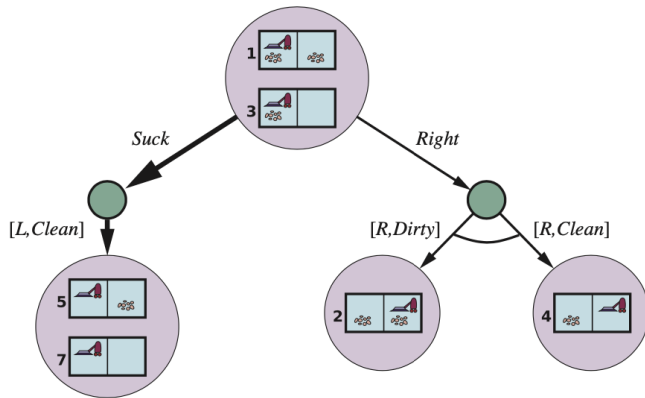
With sensors



- ▶ The agent knows where it is and sees the dirt (if any) on its spot
- ▶ The **transition model** becomes a **function** of a **belief state**, an **action**, and a **another belief state**
 - ▶ In case of nondeterminism (right), we solve the problem for every possibility. **How?**

Searching through the belief space in partially observable environments

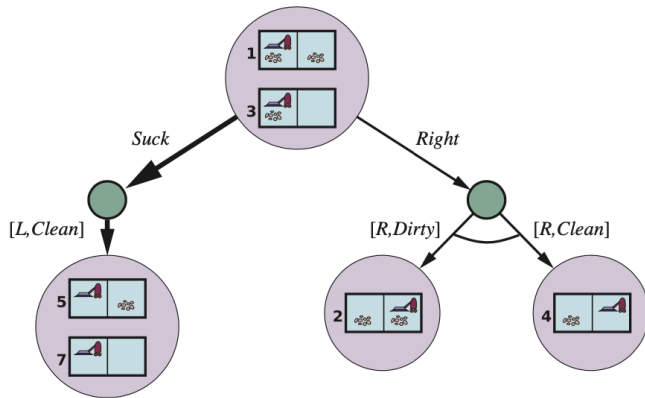
With sensors, in a nondeterministic world



► Using an **AND-OR** tree

Searching through the belief space in partially observable environments

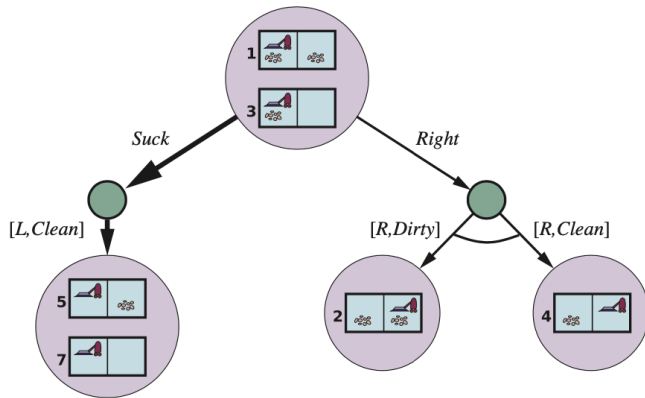
With sensors, in a nondeterministic world



- Using an **AND-OR tree**
- Notice how the nodes are now **belief states**

Searching through the belief space in partially observable environments

With sensors, in a nondeterministic world



- Using an **AND-OR** tree
- Notice how the nodes are now **belief states**
- The solution is a conditional **plan**

Online search

- So far we have looked at search algorithms which are **offline**. They compute the solution based on a graph before making a move.
- Online algorithms have to actually move around to discover the graph.
- For example, learning real-time A* (LRTA*).



End of presentation

That was it for today! Any [questions](#) or [comments](#)?

Next week you are going to be planning difficult dinner parties and playing Sudoku.
(Chapter 5)